

Department of Computer Science

University of Engineering and Technology (UET), Lahore

Data Structures and Algorithms

Title: Stack Implementation and Applications

1. Objectives

After completing this, students will be able to:

- understand the **stack data structure** and its properties
- implement stack using **arrays**
- implement stack using **linked lists**
- analyze stack operations and their **time complexity**
- implement push and pop operations using **linked list (start insertion and end insertion)**
- apply stacks in solving practical programming problems
- convert **infix expressions to postfix**
- evaluate **postfix expressions**
- reverse strings using stack

2. Introduction

A **stack** is a linear data structure that follows the **LIFO principle (Last In First Out)**.

This means the last element inserted into the stack is the first one to be removed.

Stacks are widely used in many computing applications such as:

- expression evaluation
- recursion
- compiler syntax checking
- undo/redo operations

- browser history navigation
- memory management in programming languages

3. Stack Operations

The basic operations performed on a stack are:

Operation	Description
Push	Inserts an element into the stack
Pop	Removes the top element from the stack
Peek / Top	Returns the top element without removing it
isEmpty	Checks if the stack is empty
isFull	Checks if the stack is full (for array implementation)

4. Stack Implementation using Array

In array implementation, a stack is represented using a fixed-size array.

A variable called **top** keeps track of the index of the current top element.

Algorithm: PUSH Operation

1. Check if stack is full
2. If full → display **Stack Overflow**
3. Otherwise
 - increment top
 - insert element at stack[top]

Algorithm: POP Operation

1. Check if stack is empty
2. If empty → display **Stack Underflow**
3. Otherwise
 - retrieve element from stack[top]

- decrement top

Program: Stack using Array (C++)

```
1. #include <iostream> using namespace std;

#define MAX 100

class Stack {
    int arr[MAX];
    int top;

public:
    Stack() {
        top = -1;
    }

    void push(int x) {
        if (top == MAX - 1) {
            cout << "Stack Overflow\n";
            return;
        }
        arr[++top] = x;
        cout << x << " pushed into stack\n";
    }

    void pop() {
        if (top < 0) {
            cout << "Stack Underflow\n";
            return;
        }
        cout << arr[top--] << " popped\n";
    }

    void peek() {
        if (top < 0) {
            cout << "Stack Empty\n";
            return;
        }
        cout << "Top element: " << arr[top] << endl;
    }

    void display() {
        for (int i = top; i >= 0; i--)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Stack s;

    s.push(10);
    s.push(20);
    s.push(30);

    s.display();
    s.peek();
    s.pop();
    s.display();
}
```

```
    return 0;
}
2.
```

5. Stack Implementation using Linked List

A stack can also be implemented using a **linked list**, where each node contains:

- data
- pointer to next node

Unlike arrays, linked lists allow **dynamic memory allocation**.

Method 1: Push at Start (Recommended)

This is the most efficient method because insertion and deletion occur at the head of the list.

Time Complexity

Push = **O(1)**

Pop = **O(1)**

Program: Stack using Linked List (Push at Start)

```
#include <iostream> using namespace std;

class Node {
public:
    int data;
    Node* next;
};

class Stack {
    Node* top;
public:
    Stack() {
        top = NULL;
    }

    void push(int x) {
        Node* newNode = new Node();
        newNode->data = x;
        newNode->next = top;
        top = newNode;
    }

    void pop() {
        if (top == NULL) {
            cout << "Stack Underflow\n";
            return;
        }
    }
};
```

```

    }

    Node* temp = top;
    cout << temp->data << " popped\n";
    top = top->next;
    delete temp;
}

void display() {
    Node* temp = top;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
};

int main() {
    Stack s;

    s.push(5);
    s.push(10);
    s.push(15);

    s.display();

    s.pop();

    s.display();

    return 0;
}

```

Method 2: Push at End

In this method, new nodes are inserted at the **end of the linked list**.

This method is less efficient because the program must traverse the entire list before insertion.

Time Complexity

Push = $O(n)$

6. Applications of Stack

Stacks are widely used in solving various computational problems such as:

- expression conversion
- expression evaluation
- syntax parsing in compilers

- backtracking algorithms
- recursion
- string manipulation

7. Programming Tasks

Students must implement the following applications using stack.

Task 1: Reverse a String using Stack

Objective

Use stack to reverse a string.

Algorithm

1. Read a string
2. Push all characters into stack
3. Pop characters one by one
4. Construct reversed string

Program

```
#include <iostream> #include <stack> using namespace std;

int main() {

    string str;
    cout << "Enter string: ";
    cin >> str;

    stack<char> s;

    for(char c : str)
        s.push(c);

    cout << "Reversed string: ";

    while(!s.empty()) {
        cout << s.top();
        s.pop();
    }

    return 0;
}
```

Task 2: Check Balanced Parentheses

Example:

Input: $\{(a+b)*(c+d)\}$

Output: Balanced

Students must implement a program that checks whether parentheses are balanced.

Task 3: Infix to Postfix Conversion

Example

Infix expression:

$A + B * C$

Postfix expression:

$ABC*+$

Students must implement the algorithm using stack.

Task 4: Postfix Expression Evaluation

Example:

Postfix: $23*54*+9-$

Result:

17

Students must evaluate postfix expressions using stack.

Task 5: Decimal to Binary using Stack

Convert a decimal number to binary representation using stack.

Task 6: Palindrome Check using Stack

Students must check whether a string is palindrome using stack.

Example:

madam → palindrome

hello → not palindrome

8. Exercise

Students must complete the following tasks.

Part A (Basic Implementation)

1. Implement stack using **array**
2. Implement stack using **linked list**
3. Implement push operation
4. Implement pop operation
5. Implement peek operation
6. Display stack elements

Part B (Application Problems)

Students must implement:

1. Reverse string using stack
2. Balanced parentheses checker
3. Infix to postfix conversion
4. Postfix expression evaluation
5. Decimal to binary conversion

9. Complexity Analysis

Operation	Array Stack	Linked List Stack
Push	$O(1)$	$O(1)$

Pop	O(1)	O(1)
Peek	O(1)	O(1)
Memory	Fixed	Dynamic

10. Questions

1. What is stack?
2. Why is stack called LIFO?
3. What is stack overflow?
4. What is stack underflow?
5. What is difference between stack and queue?
6. Why is stack useful in expression evaluation?
7. What is difference between infix and postfix notation?
8. What is time complexity of push and pop?
9. Why is linked list stack dynamic?
10. Which stack implementation is better and why?

11. Submission Requirements

Students must submit:

- Google Class Room
- dry run of all the programs
- C++ source code
- program outputs
- complexity analysis
- answers to viva questions